Sistemas Operacionais

Coordenadoria de Informática

Sistemas de Informação

Prof. Flávio Giraldeli

Introdução à Programação Concorrente com *PThreads*

Assunto exclusivo para o Trabalho 1

Pthreads

- O que é?
  - Bibliotecas implementando o padrão de POSIX threads são normalmente chamadas de Pthreads.
  - Elas são usadas normalmente em sistemas do tipo UNIX como Linux e Solaris, mas existem implementações para Windows.
  - Basicamente, Pthreads é um padrão que define uma interface para criação e manipulação de threads.

Primitivas Básicas:

Gerenciamento

pthread_create(thread, attr, start_routine, arg)

- O que faz?
  - Essa função cria uma thread que executa a função por ela especificada.
- Parâmetros:
  - **thread**: ponteiro estrutura previamente alocada que conterá os atributos da thread
  - **attr**: estrutura contendo opções de criação para a thread (NULL usa os valores padrão)
  - **start_routine**: função que será executada pela thread
  - **arg**: argumento recebido pela função

Primitivas Básicas:

Gerenciamento

pthread_exit(retval)

- O que faz?
  - Essa função termina a execução da thread.
- Parâmetros:
  - **retval**: valor de retorno da thread

pthread_join(th, thread_return)

- O que faz?
  - Essa função faz com que a thread que a chamou espere até que a thread passada como parâmetro retorne.
- Parâmetros:
  - **th**: thread a ser esperada
  - **thread_return**: guarda o valor de retorno da thread.

Um Exemplo Simples (Uma Thread)

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
int soma; /* este dado eh compartilhado pela(s) thread(s) */
void *runner(void *param); /* threads chamam essa funcao */

int main(int argc, char *argv[]) {
    pthread_t tid; /* o identificador da thread */
    pthread_attr_t attr; /* os atributos da thread */

    pthread_attr_init(&attr); /* ajusta os atributos padrao da thread */
    pthread_create(&tid, &attr, runner, argv[1]); /* cria a thread */
    pthread_join(tid, NULL); /* espera pelo termino da thread */

    printf("soma = %d\n", soma);
}

void *runner(void *param) {    /* A thread vai executar essa funcao */
    int i, upper = atoi(param);
    soma = 0;
    for (i = 1; i <= upper; i++) soma += i;
    pthread_exit(0);
}
```

Um Exemplo Simples (Várias Threads)

```
#define NUM_THREADS 10
                ⋮
/* um array de threads a serem unidas */
pthread_t workers[NUM_THREADS];

                ⋮

/* criação das threads (também num loop for)*/

                ⋮

/* join nas threads (a thread mãe espera pelas demais)*/
for (int i = 0; i < NUM THREADS; i++)
    pthread_join(workers[i], NULL);
                ⋮
```

Acesso a Dados Compartilhados: Mutexes

- A biblioteca PThreads fornece funções para acesso exclusivo a dados compartilhados através de ***mutexes***.
- O mutex garante três coisas:
  - **Atomicidade**: o travamento de um mutex é sempre uma operação atômica, o que significa dizer que o S.O. ou a biblioteca de threads garante que se uma thread alocou (travou) o mutex , nenhuma outra thread terá sucesso se tentar travá-lo ao mesmo tempo.
  - **Singularidade**: se uma thread alocou um mutex , nenhuma outra será capaz de alocá-lo antes que a thread original libere o travamento.
  - **Sem espera ocupada**: se uma thread tenta travar um mutex que já está travado por uma primeira thread , a segunda thread ficará suspensa até que o travamento seja liberado. Nesse momento, ela será acordada e continuará a sua execução, tendo o mutex travado para si.

Primitivas Básicas:
Exclusão Mútua

pthread_mutex_init(mutex, mutexattr)

- O que faz?
  - Essa função inicializa um mutex, que implementa o paradigma de fechadura, explicado anteriormente.
- Parâmetros:
  - **mutex**: ponteiro para a estrutura previamente alocada que conterá o mutex.
  - **mutexattr**: ponteiro para a estrutura contendo opções para a criação do mutex. Caso valha NULL, valores padrão serão usados.

Primitivas Básicas:

Exclusão Mútua

pthread_mutex_lock(mutex)

- O que faz?
  - Essa função é basicamente a fechadura explicada anteriormente: caso uma thread chegue a ela e ela esteja destrancada, ela a tranca e continua executando o código após ela. Se uma thread chega a ela e ela está trancada, ela para sua execução, até que a fechadura seja eventualmente destrancada por outra thread.
- Parâmetros:
  - **mutex**: o mutex que será usado como fechadura.

Primitivas Básicas:
Exclusão Mútua

pthread_mutex_unlock(mutex)

- O que faz?
  - Essa função destranca um mutex.
- Parâmetros:
  - **mutex**: fechadura a ser destrancada

pthread_mutex_destroy(mutex)

- O que faz?
  - Essa função destrói um mutex, desalocando a memória que ele gasta.
- Parâmetros:
  - **mutex**: estrutura que será destruída

Acesso a Dados Compartilhados: **Mutexes**

Criando e Inicializando um Mutex:

```
#include <pthread.h>
pthread_mutex_t mutex; /* declaração da variável mutex */

/* cria e inicializa o mutex lock */
pthread_mutex_init(&mutex, NULL);

/* adquire o mutex lock */
pthread_mutex_lock(&mutex);

/* seção crítica*/

/* libera mutex lock */
pthread_mutex_unlock(&mutex);

/* desaloca as estruturas do mutex */
pthread_mutex_destroy(&mutex);
```

Semáforos não binários

- A API PThreads inclui funções que manipulam **variáveis condicionais** a fim de implementar o conceito de semáforos.
  - pthread_cond_init(cond, cond_attr)
  - pthread_cond_destroy(cond)
  - pthread_cond_wait(cond, mutex)
  - pthread_cond_signal(cond)
  - pthread_cond_broadcast(cond)

Semáforos não binários

- Os semáforos “de verdade” não são previstos pelo padrão Pthread. No entanto, o padrão POSIX 1003.1b-1993(POSIX.1b) define o conceito de semáforo e ele está implementado na biblioteca enviada pelo professor através do include “semaphore.h”.
- Em ambas as opções, ficará a seu cargo pesquisar o uso de tais recursos se necessário for.

Maiores Informações

- **POSIX Threads Programming**
  - https://hpc-tutorials.llnl.gov/posix/
  - Autores: Blaise Barney, Lawrence Livermore National Laboratory, UCRL-MI-133316